SIMATS ENGINEERING

CO2-ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence
COURSE CODE: CSA17

ANSWERS:

(a) The search strategy used is Greedy Best-First Search (GBFS).
It selects the unvisited neighbouring location with the lowest heuristic value h(n), where h(n) is the straight-line distance from the location to the Auditorium.
The strategy considers only the estimated distance to the goal and ignores the distance already travelled, g(n). Therefore, it may find a route that is not the shortest.

(b) The robot starts at Gate. At each step, it chooses the unvisited neighbour with the smallest straight-line distance to the Auditorium.

Step	Current location	Candidate locations and heuristic values	Selected location
1	Gate	Library = 330 m; Canteen = 240 m	Canteen
2	Canteen	Hostel = 160 m; Admin = 110 m; Library = 330 m	Admin
3	Admin	Auditorium = 0 m	Auditorium
4	Auditorium	Goal reached	Stop

Explanation:

- At Gate, Canteen is selected because (240 < 330).
- At Canteen, Admin is selected because (110 < 160 < 330).
- At Admin, the robot reaches the Auditorium, whose heuristic value is zero.

(c) The route selected is:
Gate → Canteen → Admin → Auditorium
Total distance:
200+260+120=580
This route is not guaranteed to be the shortest because Greedy Best-First Search ignores the distance travelled so far when choosing the next location.
For comparison, consider the route:
Gate → Canteen → Hostel → Auditorium
200+100+180=480

The alternative route is 100 m shorter than the greedy route. Therefore, the greedy route is not optimal for this scenario.

2.

(a) The new strategy is A* Search.

A* evaluates each location using:

$f(n)=g(n)+h(n)$

where:

- g(n) = actual distance travelled from Gate to location (n).
- h(n) = estimated straight-line distance from location (n) to the Auditorium.
- f(n) = estimated total distance of the route through (n).

A* expands the location with the lowest f(n)) value from its search frontier. It can guarantee an optimal route when the heuristic is admissible and the algorithm handles the frontier and path costs correctly. An admissible heuristic never overestimates the true remaining distance.

(b) Using the A* search frontier, the robot evaluates candidate locations as follows.

Step	**Location expanded**	**Candidate**	**Distance travelled g(n)**	**Heuristic h(n)**	**Total f(n)**
1	Gate	Library	120	330	450
1	Gate	Canteen	200	240	440
2	Canteen	Hostel	300	160	460
2	Canteen	Admin	460	110	570
3	Library	Lab	270	280	550
3	Library	Hostel	420	160	580
4	Hostel	Auditorium	480	0	480

Important frontier decisions:

1. From Gate, Canteen is selected first because its f-value is 440, lower than Library's 450.
2. After expanding Canteen, the frontier contains Library (450), Hostel (460) and Admin (570). Therefore, A* expands Library next.
3. Library adds Lab to the frontier. Hostel is then expanded because its f-value is 460.

4. From Hostel, the Auditorium is reached with a total route cost of 480 m. A* selects the goal at this cost.

The route found is:
Gate → Canteen → Hostel → Auditorium
200+100+180=480
Note: A* is a global frontier-based search. Merely selecting the lowest \(g+h\) among the immediate neighbours at every junction is a local greedy rule and does not, by itself, guarantee the shortest path.

(c) The straight-line heuristic values are safe because they do not exceed the actual shortest remaining road distances.

Location	**Heuristic h(n)**	**True shortest remaining distance**	**Safe**
Gate	400 m	480 m	Yes
Library	330 m	480 m	Yes
Canteen	240 m	280 m	Yes
Hostel	160 m	180 m	Yes

For example, the true shortest remaining distance from Canteen is 100+180=280 m, which is greater than its heuristic estimate of 240 m.
Route comparison:

Search strategy	**Route**	**Total distance**
Greedy Best-First Search (Q1)	Gate → Canteen → Admin → Auditorium	580 m
A* Search (Q2)	Gate → Canteen → Hostel → Auditorium	480 m

A* finds a route that is 100 m shorter than the greedy route. The admissible heuristic helps A* find an optimal route.

3.

(a) This is a **Constraint Satisfaction Problem (CSP)**, because every workshop must be assigned one time slot while satisfying restrictions.

Variables: W1, W2, W3, W4, W5.

Domains:

W1, W2, W5: {Morning, Afternoon, Evening}

W3: {Afternoon, Evening} (not available in Morning)

W4: {Morning, Afternoon} (hall not available in Evening)

Constraints:

$W1 \neq W2$

$W1 \neq W3$

$W2 \neq W3$

$W2 \neq W4$

$W3 \neq W5$

$W4 \neq W5$.

In addition, W3 cannot be in Morning and W4 cannot be in Evening.

(b)

Use Minimum Remaining Values (MRV): select the workshop with the fewest remaining slots. Break ties by the number of unscheduled conflicting workshops, then by workshop number. Try slots in the order Morning, Afternoon, Evening.

Initial domains:

Workshop	**Remaining slots**
W1	Morning, Afternoon, Evening
W2	Morning, Afternoon, Evening
W3	Afternoon, Evening
W4	Morning, Afternoon
W5	Morning, Afternoon, Evening

Step 1: Select W3 and assign Afternoon. W3 has two slots, tied with W4; W3 has three unscheduled conflicting workshops, compared with two for W4, so W3 is

selected.

Unscheduled workshop	Remaining slots after W3 = Afternoon
W1	Morning, Evening
W2	Morning, Evening
W4	Morning, Afternoon
W5	Morning, Evening

Afternoon is removed from W1, W2 and W5 because they conflict with W3.
Step 2: Select W2 and assign Morning. Among the tied two-slot workshops, W2 and W4 have the highest degree; W2 wins by workshop number.

Unscheduled workshop	Remaining slots after W2 = Morning
W1	Evening
W4	Afternoon
W5	Morning, Evening

W1 loses Morning because it conflicts with W2. W4 loses Morning because it conflicts with W2. W5 is unaffected by W2.
Step 3: Select W1 and assign Evening. W1 has only one remaining slot.

Unscheduled workshop	Remaining slots after W1 = Evening
W4	Afternoon
W5	Morning, Evening

W1 has no direct conflict with W4 or W5, so their domains remain unchanged.
Step 4: Select W4 and assign Afternoon. W4 has only one remaining slot.

Unscheduled workshop	Remaining slots after W4 = Afternoon
W5	Morning, Evening

W5 is unaffected by this assignment because its remaining slots are Morning and Evening.
Step 5: Select W5 and assign Morning, the first available slot in the required order.

Time slot	Workshop(s)
Morning	W2, W5
Afternoon	W3, W4
Evening	W1

(c)MRV reduces search effort by assigning the most restricted workshop first, helping expose conflicts early. If any workshop has no remaining slot, the current assignment is inconsistent. The algorithm should backtrack, undo the most recent assignment and try another slot. It continues backtracking until a valid timetable is found or all possibilities are exhausted.

4.

(a) The technique is the Minimax Algorithm. Player X is MAX and tries to maximize the score; Player O is MIN and tries to minimize X's score. It assumes both players play optimally and choose moves that best serve their objectives.

(b) Calculate the value of every node (5 marks)

Final scores from left to right:

|4, 7 | 8, 2 | 3, 5 | 6, 9|.

At the bottom level, X selects the maximum from each pair.

Final scores	X's decision	Node value
4, 7	max(4, 7)	7
8, 2	max(8, 2)	8
3, 5	max(3, 5)	5
6, 9	max(6, 9)	9

O then selects the minimum value from each pair of X-node values.

O's reply	Values available	O's decision	Node value
First reply under X's first move	7, 8	min(7, 8)	7
Second reply under X's first move	5, 9	min(5, 9)	5

At the root, X chooses max(7, 5) = 7.
Tree summary: root X (MAX) = 7; its two O (MIN) children have values 7 and 5; the four X (MAX) nodes below them have values 7, 8, 5 and 9; the terminal scores are 4, 7, 8, 2, 3, 5, 6 and 9.

(c) Best first move and guaranteed score (3 marks)

Best first move: X's first (left) move.
Guaranteed score: 7.
Optimal sequence: X chooses the left move; O chooses the first reply (value 7 rather than 8); X then selects the final move that gives a score of 7.
Therefore, the final score under optimal play is 7.

5.

(a) Alpha-Beta Pruning optimizes Minimax by avoiding branches that cannot change the final decision. α is the best value MAX can guarantee so far; β is the best (lowest) value MIN can guarantee.
Step 1 — M1: Evaluate 6, 9, 7. MIN selects min(6, 9, 7) = 6. MAX's best value so far is α = 6.
Step 2 — M2: The first score is 4, so β = 4. Since β ≤ α (4 ≤ 6), the remaining scores under M2 cannot improve the final decision. Prune 8 and 10.
Step 3 — M3: The first score is 7, then the second is 5, so β = 5. Since β ≤ α (5 ≤ 6), prune the final score 12.

Engine move	**Reply scores (left to right)**	**Best value for MIN so far**	**MAX α**	**Result**
M1	6, 9, 7	6 → 6 → 6	6	Evaluate all three
M2	4, SKIPPED (8), SKIPPED (10)	4; cutoff	6	Prune 8 and 10
M3	7, 5, SKIPPED (12)	7 → 5; cutoff	6	Prune 12

(b) M1: 3 positions evaluated. M2: 1 position evaluated. M3: 2 positions evaluated. Total = 3 + 1 + 2 = 6 of the 9 positions.
The engine chooses M1, with minimax value 6.
Without pruning:
M1 = min(6, 9, 7) = 6

M2 = min(4, 8, 10) = 4
M3 = min(7, 5, 12) = 5
Therefore, max(6, 4, 5) = 6, the same result.

(c) Move ordering affects pruning efficiency. Examining promising moves first raises α early and can allow more branches to be pruned. Poor ordering can mean fewer cutoffs and more positions evaluated.
An evaluation function estimates how favorable a position is when the engine cannot search all the way to the end of the game. Higher scores indicate better positions for MAX, allowing the engine to compare moves at the search-depth limit.